

```
from google.colab import files
uploaded_files=files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 10) DateFruit Dataset.csv to 10) DateFruit Dataset.csv

```
import pandas as pd
import numpy as np
df=pd.read_csv("10) DateFruit Dataset.csv")
df.head()
```

	AREA	PERIMETER	MAJOR_AXIS	MINOR_AXIS	ECCENTRICITY	EQDIASQ	SOLIDITY	CONVEX_AREA	EXTENT	ASPECT_RATIO	...	KurtosisRR	KurtosisRG	Kurt
0	422163	2378.908	837.8484	645.6693	0.6373	733.1539	0.9947	424428	0.7831	1.2976	...	3.2370	2.9574	
1	338136	2085.144	723.8198	595.2073	0.5690	656.1464	0.9974	339014	0.7795	1.2161	...	2.6228	2.6350	
2	526843	2647.394	940.7379	715.3638	0.6494	819.0222	0.9962	528876	0.7657	1.3150	...	3.7516	3.8611	
3	416063	2351.210	827.9804	645.2988	0.6266	727.8378	0.9948	418255	0.7759	1.2831	...	5.0401	8.6136	
4	347562	2160.354	763.9877	582.8359	0.6465	665.2291	0.9908	350797	0.7569	1.3108	...	2.7016	2.9761	

5 rows × 35 columns

```
df.shape
```

```
(898, 35)
```

```
df.describe()
```

	AREA	PERIMETER	MAJOR_AXIS	MINOR_AXIS	ECCENTRICITY	EQDIASQ	SOLIDITY	CONVEX_AREA	EXTENT	ASPECT_RATIO	...	Sk
count	898.000000	898.000000	898.000000	898.000000	898.000000	898.000000	898.000000	898.000000	898.000000	898.000000	...	898.000
mean	298295.207127	2057.660953	750.811994	495.872785	0.737468	604.577938	0.981840	303845.592428	0.736267	2.131102	...	0.250
std	107245.205337	410.012459	144.059326	114.268917	0.088727	119.593888	0.018157	108815.656947	0.053745	17.820778	...	0.635
min	1987.000000	911.828000	336.722700	2.283200	0.344800	50.298400	0.836600	2257.000000	0.512300	1.065300	...	-1.029
25%	206948.000000	1726.091500	641.068650	404.684375	0.685625	513.317075	0.978825	210022.750000	0.705875	1.373725	...	-0.196
50%	319833.000000	2196.345450	791.363400	495.054850	0.754700	638.140950	0.987300	327207.000000	0.746950	1.524150	...	0.135
75%	382573.000000	2389.716575	858.633750	589.031700	0.802150	697.930525	0.991800	388804.000000	0.775850	1.674750	...	0.595
max	546063.000000	2811.997100	1222.723000	766.453600	1.000000	833.827900	0.997400	552598.000000	0.856200	535.525700	...	3.095

8 rows × 34 columns

```
df.isnull().sum()
```



```

    0
    AREA      0
df.columns
Index(['AREA', 'PERIMETER', 'MAJOR_AXIS', 'MINOR_AXIS', 'ECCENTRICITY',
      'EQDIASQ', 'SOLIDITY', 'CONVEX_AREA', 'EXTENT', 'ASPECT_RATIO',
      'COMPACTNESS', 'SHAPEFACTOR_1', 'SHAPEFACTOR_2',
      'SHAPEFACTOR_3', 'SHAPEFACTOR_4', 'MeanRR', 'MeanRG', 'MeanRB',
      'StdDevRR', 'StdDevRG', 'StdDevRB', 'SkewRR', 'SkewRG', 'SkewRB',
      'KurtosisRR', 'KurtosisRG', 'KurtosisRB', 'EntropyRR', 'EntropyRG',
      'EntropyRB', 'ALLdaub4RR', 'ALLdaub4RG', 'ALLdaub4RB', 'Class'],
      dtype='object')
    SOLIDITY  0

```

```

x=df.drop("Class",axis=1)
y=df["Class"]

```

```

from sklearn.preprocessing import StandardScaler, LabelEncoder
le=LabelEncoder()
y=le.fit_transform(y)

```

```

SHAPEFACTOR_1  0
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)

```

```

scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)

```

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

```

```

x_train_tensor=torch.tensor(x_train, dtype=torch.float32)
y_train_tensor=torch.tensor(y_train, dtype=torch.long)
x_test_tensor=torch.tensor(x_test, dtype=torch.float32)
y_test_tensor=torch.tensor(y_test, dtype=torch.long)

```

```

SkewRB 0
train_dataset=TensorDataset(x_train_tensor,y_train_tensor)
test_dataset=TensorDataset(x_test_tensor,y_test_tensor)

```

```

train_loader=DataLoader(train_dataset,batch_size=32,shuffle=True)
test_loader=DataLoader(test_dataset,batch_size=32,shuffle=False)

```

```
class ANN(nn.Module):
    def __init__(self):
        super(ANN,self).__init__()

        self.model=nn.Sequential(
            nn.Linear(x_train.shape[1],64),
            nn.ReLU(),
            nn.Linear(64,64),
            nn.ReLU(),
            nn.Linear(64,7)
        )

    def forward(self,x):
        return self.model(x)
```

```
model=ANN()
criterion=nn.CrossEntropyLoss()
optimizer=optim.Adam(model.parameters()),lr=0.001)
```

```
epochs=100
for epoch in range(epochs):
    model.train()
    running_loss=0.0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        output=model(xb)
        loss=criterion(output,yb)
        loss.backward()
        optimizer.step()
        running_loss+=loss.item()

    train_loss=running_loss/len(train_loader)
    print(f"Epoch {epoch+1}/{epochs}, Train Loss: {train_loss:.4f}")
```

```
Epoch 1/100, Train Loss: 1.7475
Epoch 2/100, Train Loss: 1.1323
Epoch 3/100, Train Loss: 0.7209
Epoch 4/100, Train Loss: 0.5564
Epoch 5/100, Train Loss: 0.4635
Epoch 6/100, Train Loss: 0.4091
Epoch 7/100, Train Loss: 0.3630
Epoch 8/100, Train Loss: 0.3154
Epoch 9/100, Train Loss: 0.2915
Epoch 10/100, Train Loss: 0.2689
Epoch 11/100, Train Loss: 0.2447
Epoch 12/100, Train Loss: 0.2314
Epoch 13/100, Train Loss: 0.2220
```

```
Epoch 14/100, Train Loss: 0.2147
Epoch 15/100, Train Loss: 0.2154
Epoch 16/100, Train Loss: 0.1973
Epoch 17/100, Train Loss: 0.1823
Epoch 18/100, Train Loss: 0.1748
Epoch 19/100, Train Loss: 0.1654
Epoch 20/100, Train Loss: 0.1654
Epoch 21/100, Train Loss: 0.1573
Epoch 22/100, Train Loss: 0.1526
Epoch 23/100, Train Loss: 0.1497
Epoch 24/100, Train Loss: 0.1402
Epoch 25/100, Train Loss: 0.1351
Epoch 26/100, Train Loss: 0.1354
Epoch 27/100, Train Loss: 0.1353
Epoch 28/100, Train Loss: 0.1296
Epoch 29/100, Train Loss: 0.1266
Epoch 30/100, Train Loss: 0.1241
Epoch 31/100, Train Loss: 0.1170
Epoch 32/100, Train Loss: 0.1205
Epoch 33/100, Train Loss: 0.1176
Epoch 34/100, Train Loss: 0.1130
Epoch 35/100, Train Loss: 0.1152
Epoch 36/100, Train Loss: 0.1086
Epoch 37/100, Train Loss: 0.1053
Epoch 38/100, Train Loss: 0.1127
Epoch 39/100, Train Loss: 0.1018
Epoch 40/100, Train Loss: 0.0938
Epoch 41/100, Train Loss: 0.0916
Epoch 42/100, Train Loss: 0.0968
Epoch 43/100, Train Loss: 0.0945
Epoch 44/100, Train Loss: 0.0881
Epoch 45/100, Train Loss: 0.0870
Epoch 46/100, Train Loss: 0.0860
Epoch 47/100, Train Loss: 0.0800
Epoch 48/100, Train Loss: 0.0820
Epoch 49/100, Train Loss: 0.0834
Epoch 50/100, Train Loss: 0.0744
Epoch 51/100, Train Loss: 0.0764
Epoch 52/100, Train Loss: 0.0751
Epoch 53/100, Train Loss: 0.0704
Epoch 54/100, Train Loss: 0.0708
Epoch 55/100, Train Loss: 0.0687
```